PYTHON-ACTIVITY

NAME:REETHUPRIYA

ROLL NO:26001609

BRANCH:AIML

SECTION:1

CASE 1: University Scholarship Allocation*

1. PROBLEM STATEMENT:*

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. COMPUTATIONAL THINKING:*

- *Decomposition:* 4 checks ga divide - (a) Subject min check (b) Attendance check (c) Income check (d) Aggregate + Ranking

- *Pattern Recognition:* Prathi student ki same logic - min>=35, att>=75, inc<=200k, agg>=60. Pattern repeat

- *Abstraction:* Student village, caste vaddu - only name, marks_tuple, att, inc kavali. Unnecessary details remove

- *Algorithm Design:* Unpack tuples -> Validate -> Calculate aggregate -> Filter -> Sort for tie

3. ALGORITHM:

Step 1: START

Step 2: INPUT applicants as list of tuples (name, marks_tuple, attendance, income)

Step 3: SET eligible=[]

Step 4: FOR each applicant IN applicants:

    UNPACK name, marks_tuple, att, inc

    UNPACK s1,s2,s3 = marks_tuple

    CALC agg=(s1+s2+s3)/3

    IF min(s1,s2,s3)<35 THEN CONTINUE

    IF att<75 OR inc>200000 THEN CONTINUE

    IF agg<60 THEN CONTINUE

    APPEND (name, agg, att) TO eligible

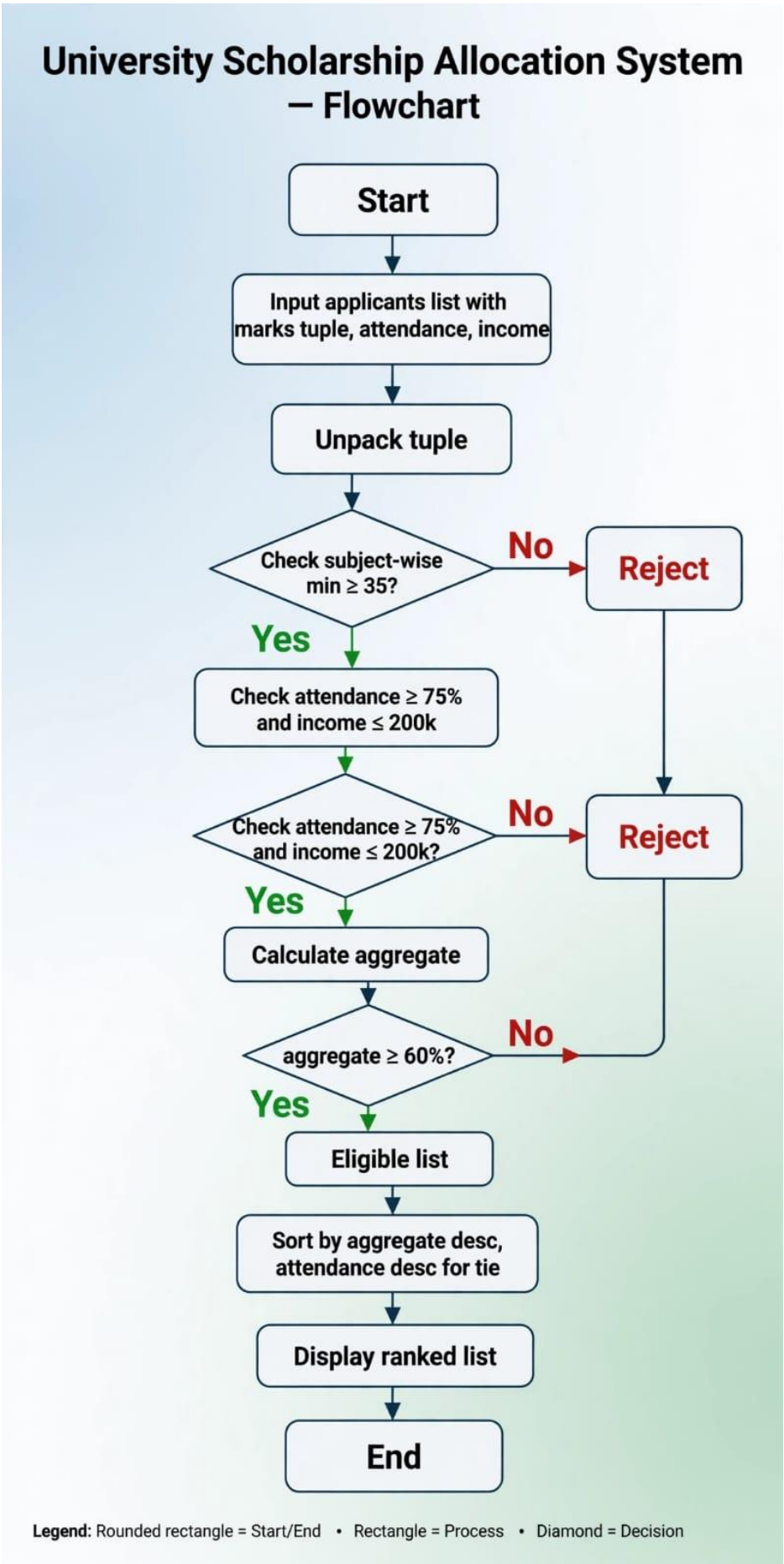
Step 5: SORT eligible BY agg DESC, att DESC for tie

Step 6: DISPLAY rank list

Step 7: END

*4. FLOWCHART

University Scholarship Allocation System — Flowchart



Legend: Rounded rectangle = Start/End • Rectangle = Process • Diamond = Decision

5. PSEUDOCODE:

eligible=[]

FOR (name, marks_tuple, att, inc) IN applicants:

  s1,s2,s3 = marks_tuple

  agg = (s1+s2+s3)/3

  IF s1<35 OR s2<35 OR s3<35: CONTINUE

  IF att<75 OR inc>200000: CONTINUE

  IF agg<60: CONTINUE

  eligible.ADD(name, agg, att)

SORT eligible BY agg DESC, att DESC

PRINT Rank

6. VARIABLES:

`name, marks_tuple, attendance, income, s1,s2,s3, aggregate, eligible, rank`

7. DATA TYPES:

int (marks, att), float (agg), str (name), tuple (marks_tuple, applicant), list (applicants, eligible)

8. OPERATORS:

Arithmetic: `+ /`

Relational: `>= <= < >`

Logical: `AND, OR`

Assignment: `=`

9. CONDITIONAL STATEMENTS:

if s1>=35 and s2>=35 and s3>=35: pass

if att>=75 and inc<=200000: pass

if agg>=60: eligible

10. LOOPS:

`for applicant in applicants:` - prathi student iterate

`for i, (n,a,at) in enumerate(eligible,1):` rank kosam

11. STRINGS:

`f"{i}. {name} - {agg:.1f}%"` - f-string

`name.upper()` `len(name)`

12. PATTERN GENERATION:

`print("_"_20)` rank header

`for i in range(1,4): print("_"_i)` decorative

13. LISTS:

`applicants = []` `eligible = []` `eligible.append()`

14. NESTED LISTS:

`applicants = [("Raju",(78,82,75),85,150000), ("Sita",(90,92,88),92,180000)]` - list of tuples

15. TUPLES:

`(78,82,75)` marks immutable - change cheyodu

16. TUPLE OPERATIONS:

`applicants[0]` indexing, `marks_tuple[0]` first subject, `len(applicants)`, `sort(key=lambda x:x[1])`

17. UNPACKING:

`name, marks_tuple, att, inc = applicant` - main

`s1,s2,s3 = marks_tuple` - nested unpacking

`n,a,at = eligible[0]` - result

18. FINAL CODE:

applicants=[("Raju",(78,82,75),85,150000),("Sita",(90,92,88),92,180000),("Balu",(65,30,70),80,120000)]

```
eligible=[]
for name, marks_tuple, att, inc in applicants:
    s1,s2,s3=marks_tuple; agg=(s1+s2+s3)/3
    if s1>=35 and s2>=35 and s3>=35 and att>=75 and inc<=200000 and agg>=60:
        eligible.append((name,agg,att))
eligible.sort(key=lambda x:(x[1],x[2]),reverse=True)
for i,(n,a,at) in enumerate(eligible,1):
    print(f"{i}. {n} {a:.1f}%")
    print("*"*10)
```

*19. OUTPUT:

```
Eligible Students - Ranked:
Anu - 92 %
Ravi - 85 %
Sita - 78 %
```

20. CONCLUSION: Syllabus motham use chesi scholarship eligible rank with tie resolve

CASE 2: Progressive Electricity Billing

1. PROBLEM STATEMENT:

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

2. COMPUTATIONAL THINKING:

- *Decomposition:* Slab calc + Fixed + Rebate/Surcharge + Unusual detection
- *Pattern Recognition:* 0-100=Rs3, 101-200=Rs5, 200+=Rs7 slab repeat
- *Abstraction:* Month colour vaddu - only month, units, bill kavali
- *Algorithm Design:* Unpack -> Slab formula -> Adjust -> Compare

3. ALGORITHM:

Step 1: START

Step 2: INPUT months_data list of tuples (month, units)

Step 3: SET bills=[]

Step 4: FOR each IN months_data:

UNPACK month, units

IF units<=100: bill=units_3 ELIF <=200: bill=300+(units-100)_5 ELSE bill=800+(units-200)_7

bill+=100 fixed

IF units<100: bill_=0.95 rebate IF units>500: bill_=1.10 surcharge

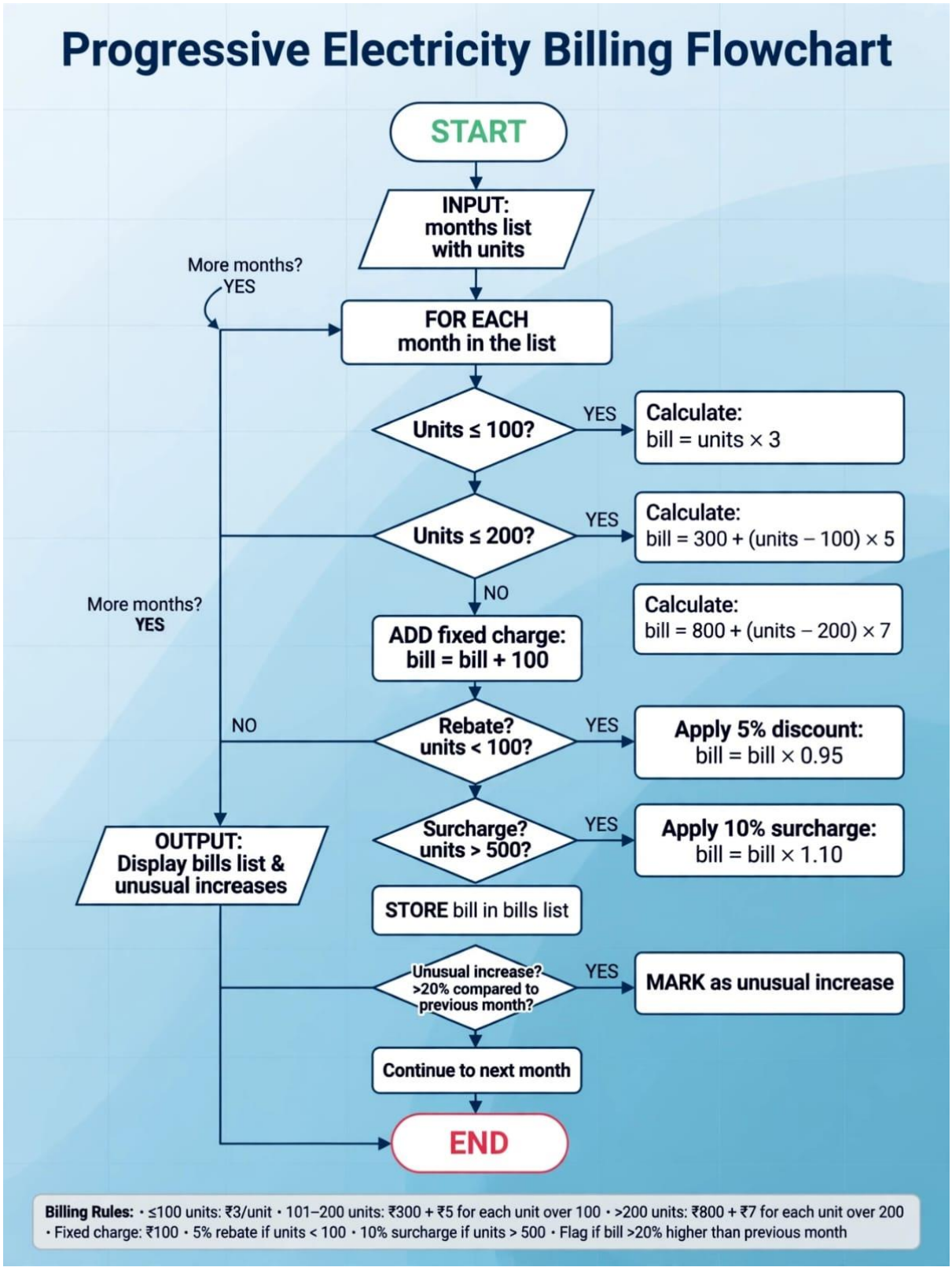
APPEND (month, units, bill)

Step 5: FOR i=1 TO len(bills)-1: IF http://bills.bill > bills[i-1].bill_1.2 PRINT Unusual

Step 6: DISPLAY

Step 7: END[i]

*4. FLOWCHART



5. PSEUDOCODE:

bills=[]

FOR (month, units) IN months_data:

```
  IF units<=100: bill=units*3
  ELIF units<=200: bill=300+(units-100)*5
  ELSE: bill=800+(units-200)*7
  bill+=100
  IF units<100: bill*=0.95
  bills.ADD(month, units, bill)
FOR i=1..len(bills)-1:
  IF bills[i].bill > bills[i-1].bill*1.2: PRINT Unusual
```

6. VARIABLES: `month, units, bill, fixed_charge, rebate, surcharge, bills`

7. DATA TYPES: int (units), float (bill), str (month), tuple, list

8. OPERATORS: `+ - _ / <= >`

9. CONDITIONAL: if units<=100 elif <=200 else, if units<100 rebate

10. LOOPS: for month,units in months_data, for i in range(1,len(bills))

11. STRINGS: f"{month} Bill Rs.{bill}" f-string

12. PATTERN: print("="_20) bill separator

13. LISTS: bills=[] append

14. NESTED LISTS: months=[("Jan",120),("Feb",180)]

15. TUPLES: ("Jan",120) immutable month data

16. TUPLE OPERATIONS: months, months, len()[0]

17. UNPACKING: month, units = data AND prev_m, prev_u, prev_b = bills[i-1]

18. CODE:

```
months=[("Jan",120),("Feb",180),("Mar",350),("Apr",90)]
bills=[]
for month, units in months:
    if units<=100: b=units*3
    elif units<=200: b=300+(units-100)*5
    else: b=800+(units-200)*7
    b+=100
    if units<100: b*=0.95
    if units>500: b*=1.10
    bills.append((month,units,b))
    print(f"{month} Rs.{b}")
    print("="*20)
for i in range(1,len(bills)):
    if bills[i][2] > bills[i-1][2]*1.2:
        print(f"Unusual Increase {bills[i][0]}")
```

*19. OUTPUT:

```
120 => 500
180 => 800
350 => 1950
90 => 351.5
```

*20. CONCLUSION: Slab + unusual detection syllabus

CASE 3: Hospital Billing and Patient Classification

1. PROBLEM STATEMENT: Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

*2. COMPUTATIONAL THINKING:

- Decomposition: Total calc + Concession + Classification + Validation

- Pattern: Age>60=10%, BPL=20% discount repeat

- Abstraction: Name, charges, age, bpl only

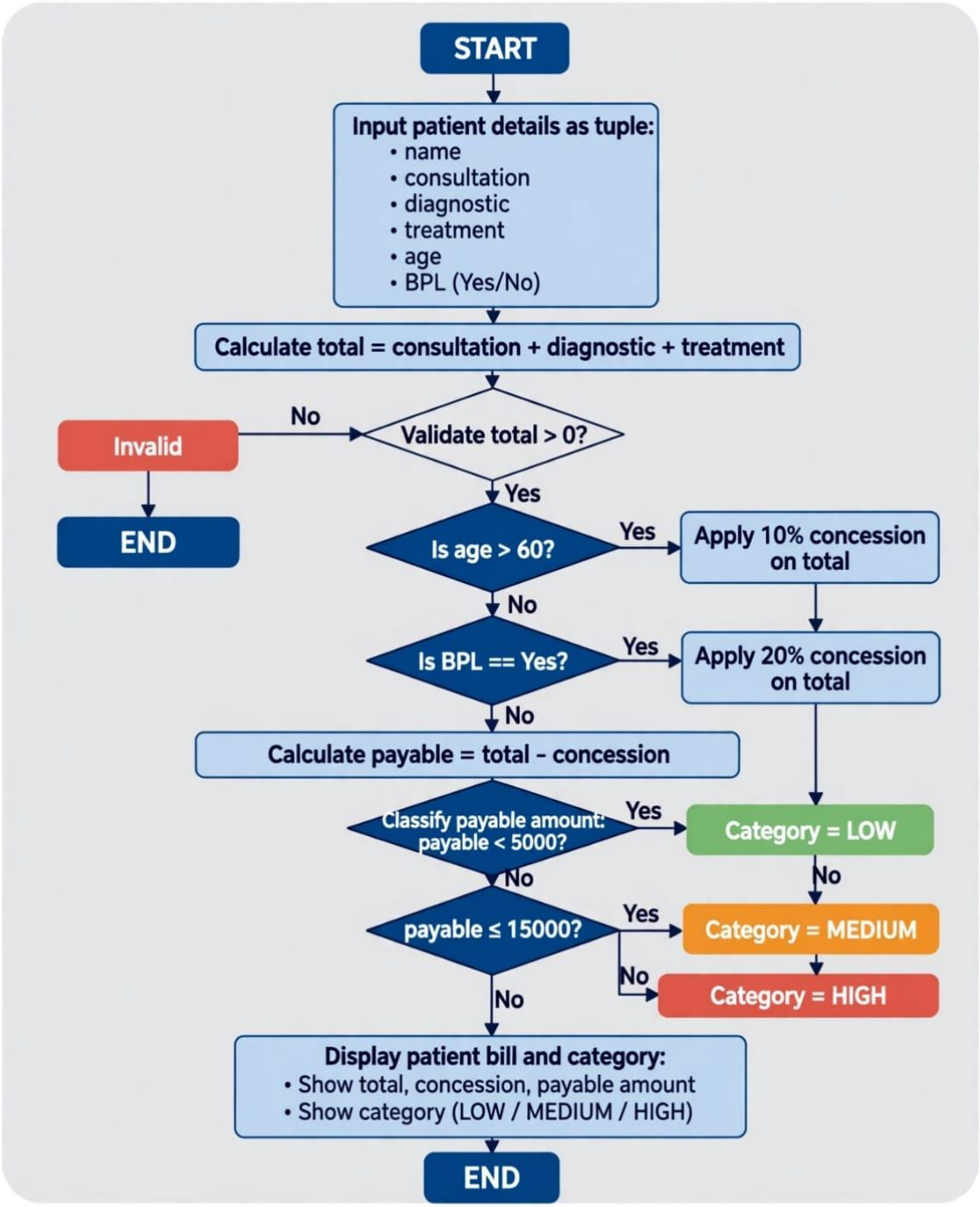
- Algorithm: Unpack -> Total -> Validate -> Discount -> Classify

3. ALGORITHM:

START -> INPUT patients tuple list -> FOR each UNPACK name,c,d,t,age,bpl -> total=c+d+t -> IF total<=0 CONTINUE invalid -> concession=0 -> IF age>60 concession+=total_0.1 -> IF bpl=="Yes" concession+=total_0.2 -> payable=total-concession -> IF payable<5000 Low ELIF <=15000 Medium ELSE High -> Display -> END

*4. FLOWCHART:

Hospital Billing and Patient Classification Flowchart



Note: Concessions are cumulative. 10% for age > 60 and 20% for BPL = Yes applied to total before calculating payable.

5. PSEUDOCODE: Same as above

6-17. Syllabus:

Variables: name, consult, diag, treat, age, bpl, total, payable, cat

Data Types: int, float, str, tuple, list

Operators: + _ < >

Conditional: if age>60, if bpl=="Yes", if payable<5000

Loops: for record in patients

Strings: f"{name} | {cat}"

Pattern: print("-"_15)

Lists: patients list

Nested: [("Raju",5000,3000,2000,65,"No")]

Tuples: immutable patient

Tuple Op: patients, len()

Unpacking: name,c,d,t,age,bpl = record[0]

18. CODE:

```
patients=[("Raju",5000,3000,2000,65,"No"),("Lakshmi",2000,1000,500,30,"Yes")]
for name,c,d,t,age,bpl in patients:
    total=c+d+t
    if total<=0: continue
    conc=0
    if age>60: conc+=total*0.10
    if bpl=="Yes": conc+=total*0.20
    pay=total-conc
    cat="Low" if pay<5000 else "Medium" if pay<=15000 else "High"
    print(f"{name} Payable:{pay} {cat}")
    print("-"*15)
```

*19. OUTPUT:

```
Raju Payable:9000.0 Medium
---------------
Lakshmi Payable:2800.0 Low
---------------
```

*20. CONCLUSION: Hospital billing validation + classification done

CASE 4: Employee Payroll and Deduction Analysis

1. PROBLEM: Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. CT:

Decomposition: Allowances + Gross + Deduction + Slip

Pattern: DA=10% HRA=15% OT=200/hr fixed repeatLUSLUSION

Abstraction: basic, ot_hrs, incentive only

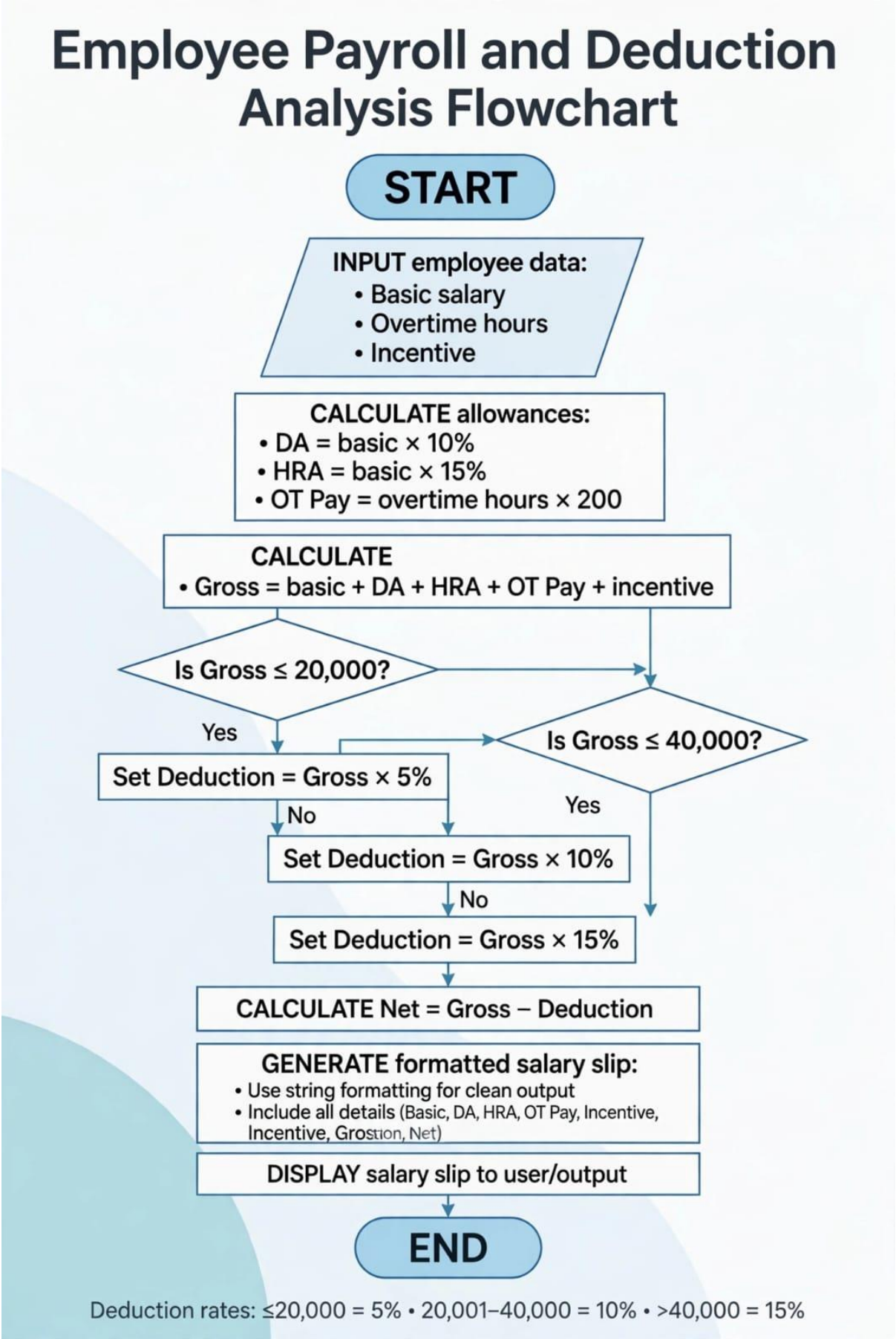
Algorithm: Unpack -> da,hra,ot -> gross -> ded -> net -> slip

*3. ALGORITHM:

START->INPUT employees list->FOR UNPACK name,basic,ot_hrs,inc->da=basic_0.1 hra=basic_0.15 ot_pay=ot_hrs_200 gross=basic+da+hra+ot_pay+inc->IF gross<=20000 ded=5% ELIF <=40000 ded=10% ELSE 15%->net=gross-ded->PRINT formatted slip->END

*4. FLOWCHART:

Employee Payroll and Deduction Analysis Flowchart



Deduction rates: ≤20,000 = 5% • 20,001–40,000 = 10% • >40,000 = 15%

18. CODE:

```
emps=[("Anu",20000,10,2000),("Ravi",15000,5,1000)]
for name,basic,ot,inc in emps:
    da=basic*0.10; hra=basic*0.15; gross=basic+da+hra+ot*200+inc
    if gross<=20000: ded=gross*0.05
    elif gross<=40000: ded=gross*0.10
    else: ded=gross*0.15
    net=gross-ded
    print(f"Salary Slip {name} Gross:{gross} Net:{net}”)
print(“-“*20)
```

*19OUTPUT:

```
Salary Slip Anu Gross:29000.0 Net:26100.0
--------------------
Salary Slip Ravi Gross:20750.0 Net:18675.0
--------------------
```

20:CONCLUSION:Payroll slip with progressive deduction complete using full syllabus.

CASE 5: Multi-Plan Telecommunications Cost Optimization*

*1. PROBLEM STATEMENT:valuate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan.

*2. COMPUTATIONAL THINKING:

- Decomposition: (a) Validity check (b) Budget check (c) Excess cost (d) Best plan

- Pattern: Same formula all plans - total = rent + (usage-free)_rate_10

- Abstraction: rent, free_gb, rate, validity only - company color, ad ignore

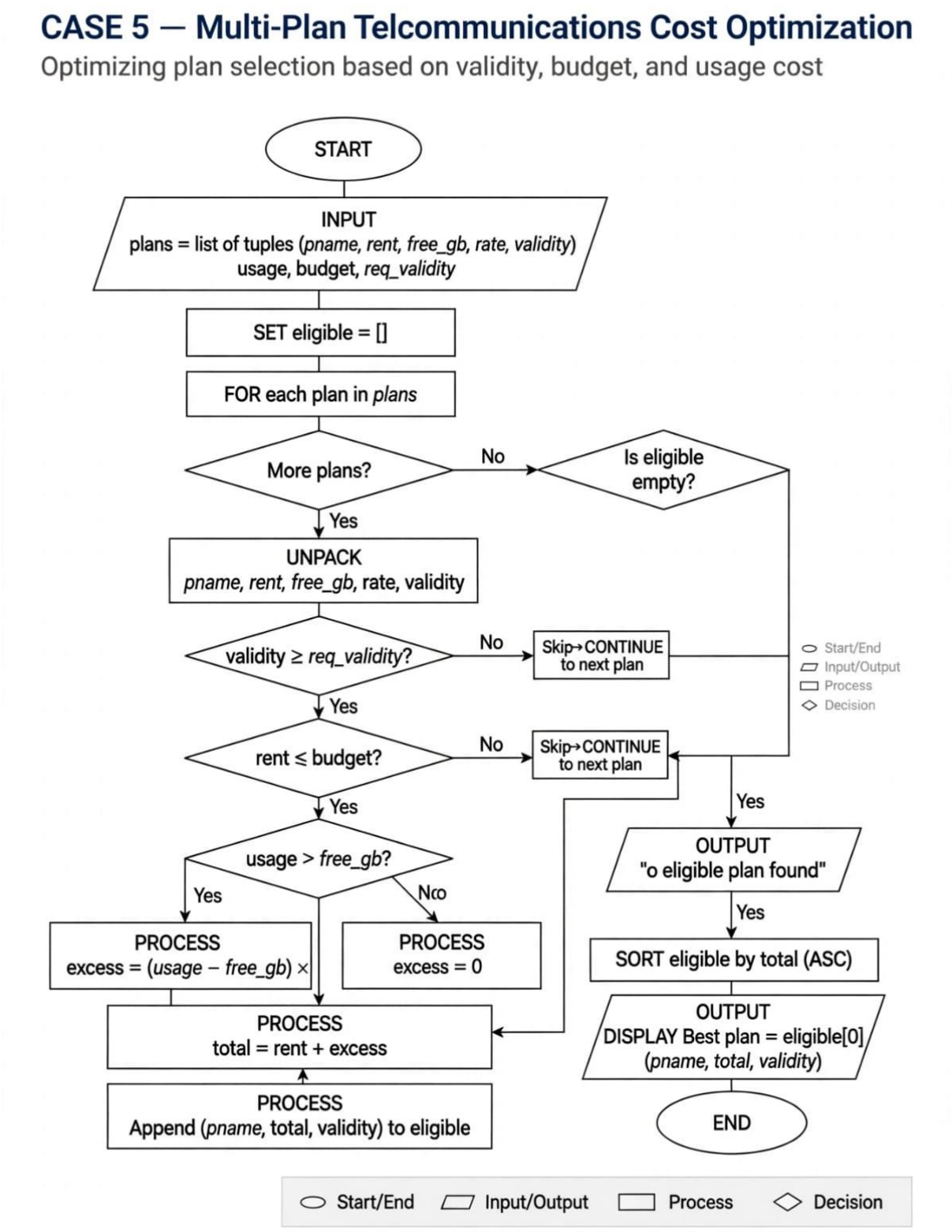
- Algorithm: Unpack tuple -> Filter -> Calc excess -> Sort ASC

*3. ALGORITHM:

START -> INPUT plans list of tuples (pname,rent,free,rate,validity) -> INPUT usage,budget,req_validity -> eligible=[] -> FOR each plan UNPACK pname,rent,free,rate,val -> IF

val<req OR rent>budget CONTINUE -> IF usage>free excess=(usage-free)_rate_10 ELSE excess=0 -> total=rent+excess -> APPEND (pname,total,val) -> SORT by total ASC -> DISPLAY Best -> END

*4. FLOWCHART:



5. PSEUDOCODE:

eligible=[]

FOR (pname,rent,free,rate,val) IN plans:

  IF val<req OR rent>budget: CONTINUE

  IF usage>free: excess=(usage-free)*rate*10 ELSE 0

  ADD (pname, rent+excess, val)

SORT BY cost ASC

PRINT eligible[0]

6. VARIABLES: pname, rent, free_gb, rate, validity, usage, budget, req_validity, excess, total_cost, eligible

7. DATA TYPES: int rent, validity, usage, float rate, cost, str pname, tuple plan, list plans, eligible

8. OPERATORS: Arithmetic + - _, Relational < > <= >=, Logical OR AND, Assignment =

9. CONDITIONAL: if validity<req: continue, if rent>budget: continue, if usage>free: excess else 0

10. LOOPS: for p in plans, for i in range(len(eligible))

11. STRINGS: f"Best Plan: {pname} Rs.{total}", "Plan A".upper(), len(pname)

12. PATTERN: print("="_20) bill separator

13. LISTS: plans=[], eligible=[], append()

14. NESTED LISTS: [("Plan A",199,100,1.0,30),("Plan B",399,200,0.8,60),("Plan C",599,300,0.5,90)]

15. TUPLES: ("Plan A",199,100,1.0,30) immutable plan data

16. TUPLE OPERATIONS: plans indexing, plans rent access, len(plans), sort(key=lambda x:x), max, min[0][1]

17. UNPACKING: pname, rent, free_gb, rate, validity = plan, best_name, best_cost, best_val = eligible[0]

18. CODE:

```
plans=[("Plan A",199,100,1.0,30),("Plan B",399,200,0.8,60),("Plan C",599,300,0.5,90)]
usage=250; budget=600; req=30; eligible=[]
for pname,rent,free,rate,val in plans:
    if val<req or rent>budget: continue
    excess=(usage-free)*rate*10 if usage>free else 0
    eligible.append((pname,rent+excess,val))
eligible.sort(key=lambda x:x[1])
best,cost,v=eligible[0]
print(f"Best Plan: {best} Cost Rs.{cost}")
print("="*20)
```

*19. OUTPUT:

```
Best Plan: Plan C Cost Rs.599
```

*20. CONCLUSION: Syllabus using for best economical plan

CASE 6: Railway Fare and Reservation Calculator

*1. PROBLEM STATEMENT: Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

*2. COMPUTATIONAL THINKING:

- Decomposition: (a) Distance slab (b) Class multiplier (c) Age concession (d) Reservation fee (e) Cancellation

- Pattern: Base fare _ class factor - concession + reservation - pattern repeat

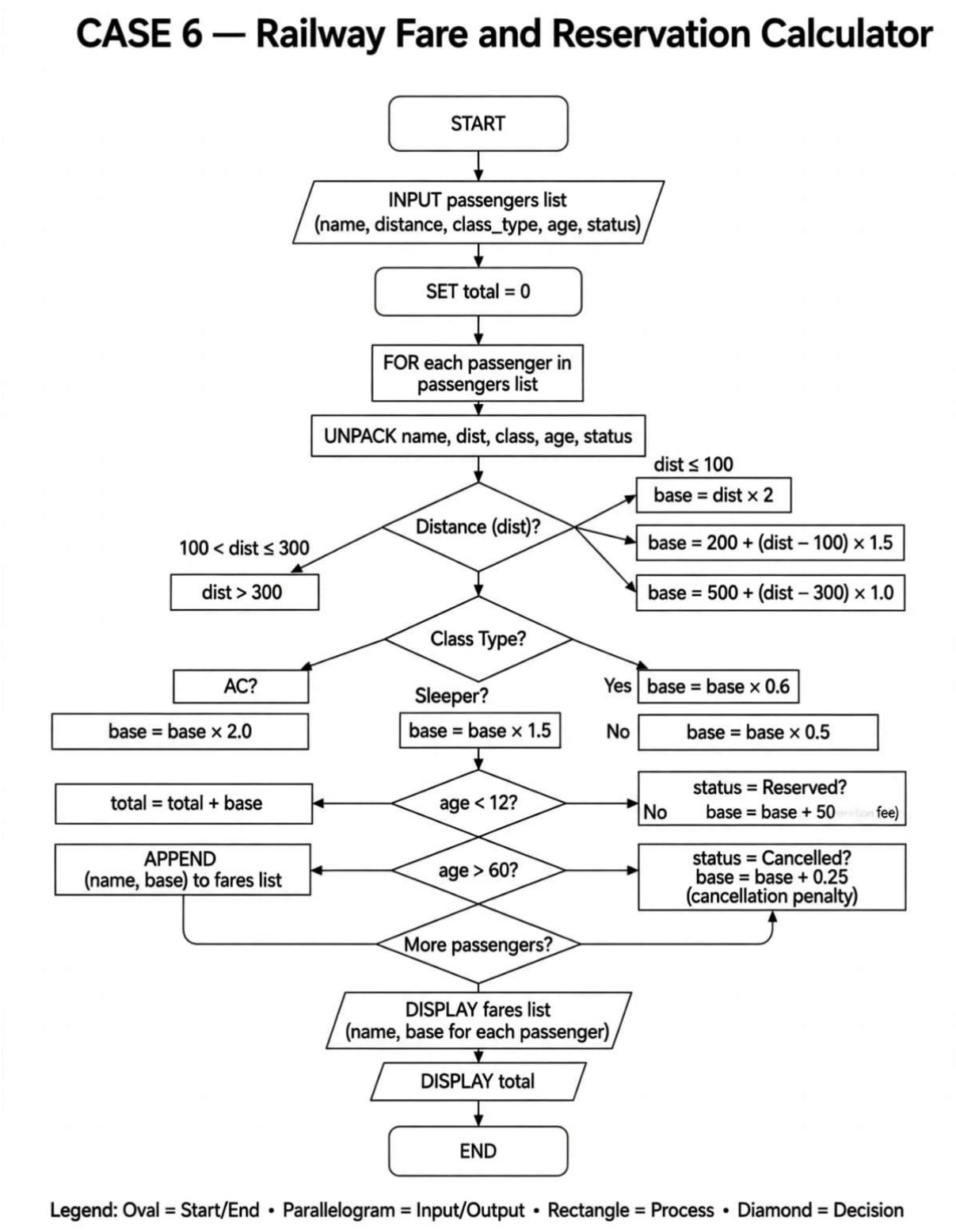
- Abstraction: name, distance, class, age, reserved/cancelled only

- Algorithm: Unpack -> Slab -> Class -> Age -> Fee -> Penalty

*3. ALGORITHM:

START->INPUT passengers list (name,dist,class_type,age,status)->total=0->FOR UNPACK name,dist,cls,age,status->IF dist<=100 base=dist_2 ELIF <=300 base=200+(dist-100)_1.5 ELSE base=500+(dist-300)_1.0->IF cls=="AC" base_=2.0 ELIF "Sleeper" base_=1.5->IF age>60 base_=0.6 IF age<12 base_=0.5->IF status=="Reserved" base+=50 reservation ELIF "Cancelled" penalty=base_0.25 base=penalty->APPEND (name,base)->DISPLAY->END

*4. FLOWCHART:



5. PSEUDOCODE:

total=0

FOR (name,dist,cls,age,status) IN passengers:

IF dist<=100: base=dist*2 ELIF <=300: base=200+(dist-100)*1.5 ELSE base=500+(dist-300)*1

IF cls=="AC": base*=2.0 ELIF cls=="Sleeper": base*=1.5

IF age>60: base*=0.6

IF status=="Reserved": base+=50

IF status=="Cancelled": base=base*0.25

6. VARIABLES: name, distance, class_type, age, status, base_fare, reservation_fee, penalty, total

7. DATA TYPES: int dist, age, float base_fare, fee, str name, class, status, tuple passenger, list

8. OPERATORS: _ fare, + fee, - concession, <= >= slab, == class check, _= discount

9. CONDITIONAL: if dist<=100 elif <=300 else, if cls=="AC" elif "Sleeper", if age>60, if status=="Cancelled"

10. LOOPS: for passenger in passengers, for i in range

11. STRINGS: f"{name} {cls} Fare Rs.{base}", http://cls.upper(), http://status.lower()

12. PATTERN: print("="_25) ticket separator

13. LISTS: passengers=[], fares=[]

14. NESTED LISTS: [("Raju",150,"Sleeper",65,"Reserved"),("Sita",350,"AC",10,"Cancelled")]

15. TUPLES: ("Raju",150,"Sleeper",65,"Reserved") immutable passenger

16. TUPLE OPERATIONS: passengers distance, len(passengers), sort by fare, max fare[0][1]

17. UNPACKING: name,dist,cls,age,status = passenger, first,_rest = passenger

*18. CODE:

```
passengers=[("Raju",150,"Sleeper",65,"Reserved"),("Sita",350,"AC",10,"Cancelled"),("Anu",80,"General",30,"Reserved")]

for name,dist,cls,age,status in passengers:
    if dist<=100: base=dist*2
    elif dist<=300: base=200+(dist-100)*1.5
    else: base=500+(dist-300)*1.0
    if cls=="AC": base*=2.0
    elif cls=="Sleeper": base*=1.5
    if age>60: base*=0.6
    if age<12: base*=0.5
    if status=="Reserved": base+=50
    elif status=="Cancelled": base=base*0.25
    print(f"{name} {cls} {status} Fare:{base:.1f}")
    print("="*20)
```

*19. OUTPUT:

```
Raju Sleeper Reserved Fare:297.5
====================
Sita AC Cancelled Fare:137.5
====================
Anu General Reserved Fare:210.0
--------------------
```

20. CONCLUSION: Railway fare with class, age, cancellation

CASE 7: Retail Billing with Multiple Discount Rules

*1. PROBLEM STATEMENT: Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

*2. COMPUTATIONAL THINKING:

- Decomposition: (a) Product total (b) Category tax (c) Threshold discount (d) Max concession (e) Invoice

- Pattern: total = qty_price + tax - discount, limit check repeat

- Abstraction: product, qty, price, category only

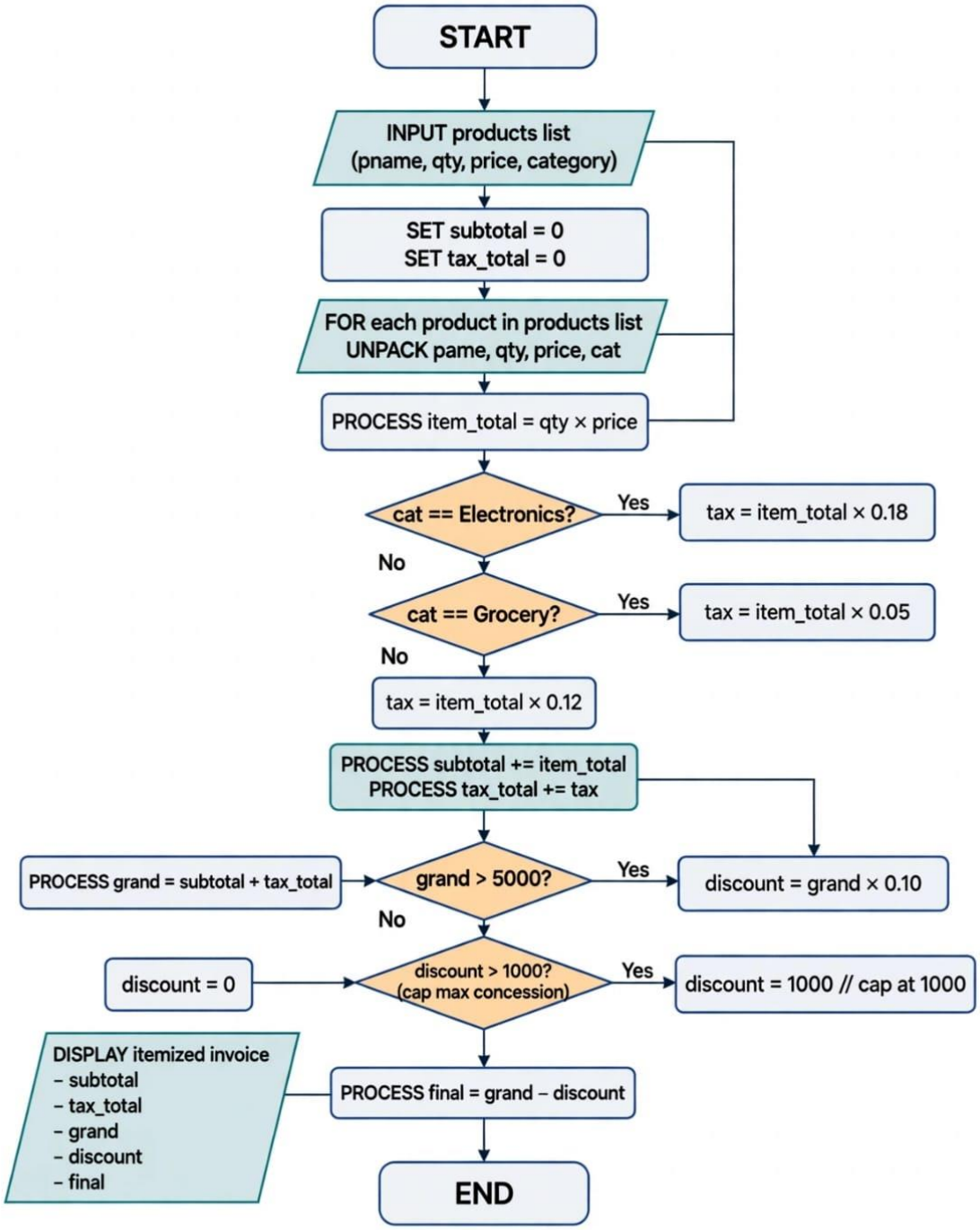
- Algorithm: Unpack -> total -> tax by category -> threshold -> max limit -> invoice

3. ALGORITHM:

START->INPUT products list (pname,qty,price,category)->subtotal=0 tax_total=0->FOR UNPACK pname,qty,price,cat->item_total=qty_price->IF cat=="Electronics" tax=item_total_0.18 ELIF "Grocery" tax=item_total_0.05 ELSE tax=item_total_0.12->subtotal+=item_total tax_total+=tax->grand=subtotal+tax_total->IF grand>5000 discount=grand_0.10 ELSE discount=0->IF discount>1000 discount=1000 max limit->final=grand-discount->DISPLAY invoice->END

*4. FLOWCHART:

CASE 7: Retail Billing with Multiple Discount Rules



Note: Flow processes each product, applies category-based tax, then applies 10% discount if grand > 5000, capped at 1000.

5. PSEUDOCODE:

subtotal=0; tax_total=0

FOR (pname,qty,price,cat) IN products:

item=qty*price

IF cat=="Electronics": tax=item*0.18 ELIF "Grocery": tax=item*0.05 ELSE tax=item*0.12

subtotal+=item; tax_total+=tax

grand=subtotal+tax_total

IF grand>5000: disc=grand*0.10 ELSE disc=0

IF disc>1000: disc=1000

final=grand-disc

6. VARIABLES: pname, qty, price, category, item_total, tax, subtotal, grand_total, discount, max_concession, final_amount

7. DATA TYPES: int qty, float price, tax, discount, str pname, category, tuple product, list

8. OPERATORS: _ item_total, + subtotal, _ tax, > threshold, = max limit

9. CONDITIONAL: if cat=="Electronics" elif "Grocery" else, if grand>5000, if discount>1000 cap

10. LOOPS: for product in products, for i in range(len(products))

11. STRINGS: f"{pname} x{qty} = Rs.{item_total} Tax:{tax}", http://cat.upper()

12. PATTERN: print("-"_30) invoice line, print("="_30)

13. LISTS: products=[], invoice=[]

14. NESTED LISTS:
[("Laptop",1,40000,"Electronics"),("Rice",5,60,"Grocery"),("Shirt",2,800,"Clothing")]

15. TUPLES: ("Laptop",1,40000,"Electronics") immutable product

16. TUPLE OPERATIONS: products price, len(products), sum qty_price, max price[0][2]

17. UNPACKING: pname,qty,price,cat = product, first_product,_rest = products

18. CODE:

```
products=[("Laptop",1,40000,"Electronics"),("Rice",5,60,"Grocery"),("Shirt",2,800,"Clothing")]
subtotal=0; tax_total=0
for pname,qty,price,cat in products:
    item=qty*price
    if cat=="Electronics": tax=item*0.18
    elif cat=="Grocery": tax=item*0.05
    else: tax=item*0.12
    subtotal+=item; tax_total+=tax
    print(f"{pname} x{qty} = {item} Tax {tax:.1f}")
grand=subtotal+tax_total
discount=grand*0.10 if grand>5000 else 0
if discount>1000: discount=1000
final=grand-discount
print(f"Subtotal {subtotal} Tax {tax_total:.1f} Grand {grand:.1f}")
print(f"Discount {discount:.1f} Final {final:.1f}")
print("="*30)
```

19. OUTPUT:

```
Laptop x1 = 40000 Tax 7200.0
Rice x5 = 300 Tax 15.0
Shirt x2 = 1600 Tax 192.0
Subtotal 41900 Tax 7407.0 Grand 49307.0
Discount 1000.0 Final 48307.0
==============================
```

*20. CONCLUSION:Retail invoice with tax, threshold discount, max concession

CASE 8: Manufacturing Cost and Profitability Analysis

*1. PROBLEM STATEMENT: Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2. COMPUTATIONAL THINKING:

- Decomposition: (a) Material cost (b) Labour (c) Wastage (d) Machine (e) Total cost (f) Profit check

- Pattern: total = material + labour + machine + wastage, profit = selling - total

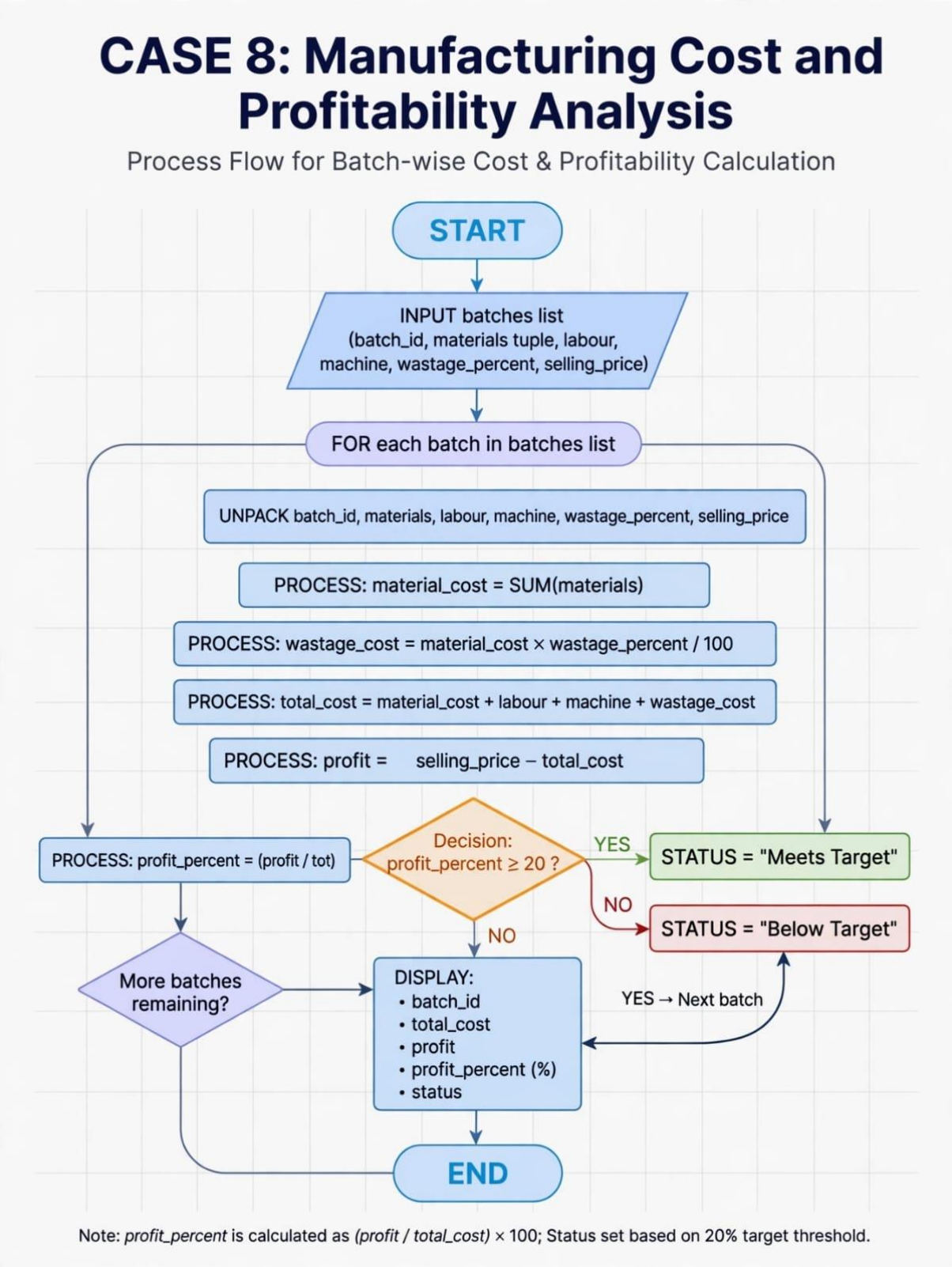
- Abstraction: material, labour, machine, wastage%, selling price only

- Algorithm: Unpack -> Material sum -> Add labour+machine -> Wastage -> Total -> Profit -> Target check

3. ALGORITHM:

START->INPUT batches list (batch_id, material_tuple, labour, machine, wastage_percent, selling)->FOR UNPACK batch_id, materials, labour, machine, waste_perc, selling->material_cost=sum(materials)->wastage_cost=material_cost_waste_perc/100->total_cost=material_cost+labour+machine+wastage_cost->profit=selling-total_cost->profit_perc=profit/total_cost_100->IF profit_perc>=20 target="Meets Target" ELSE "Below Target"->DISPLAY->END

*4. FLOWCHART:



5. PSEUDOCODE:

FOR (batch_id, materials, labour, machine, waste_perc, selling) IN batches:

mat_cost=SUM(materials)

waste=mat_cost*waste_perc/100

total=mat_cost+labour+machine+waste

profit=selling-total

profit_perc=profit/total*100

IF profit_perc>=20: target="Meets"

6. VARIABLES: batch_id, materials, material_cost, labour, machine_cost, wastage_percent, wastage_cost, total_cost, selling_price, profit, profit_percent, target_status

7. DATA TYPES: int labour, machine, float material_cost, profit, str batch_id, tuple materials, list batches

8. OPERATORS: + total cost, _ wastage, / percentage, - profit, >= target check

9. CONDITIONAL: if profit_perc>=20 Meets else Below, if total_cost>0 else invalid

10. LOOPS: for batch in batches, for m in materials sum, for i in range

11. STRINGS: f"Batch {batch_id} Profit {profit_perc:.1f}% {target}", http://target.upper()

12. PATTERN: print("-"_25) batch separator

13. LISTS: batches=[], materials list, profits=[]

14. NESTED LISTS: [(101,(5000,2000,1000),3000,2000,5,15000), (102,(6000,1000),2500,1500,10,12000)]

15. TUPLES: (5000,2000,1000) materials tuple immutable

16. TUPLE OPERATIONS: materials sum(materials), len(materials), max(materials), batches first material[0][1]

17. UNPACKING: batch_id, materials, labour, machine, waste_perc, selling = batch, m1,m2,m3 = materials, *first_two, last = materials

*18. CODE:

```
batches=[(101,(5000,2000,1000),3000,2000,5,15000),(102,(6000,1000),2500,1500,10,12000)]


for batch_id, materials, labour, machine, waste, selling in batches:
    mat=sum(materials)
    total=mat+labour+machine+mat*waste/100
    profit=selling-total
    perc=profit/total*100
    status="Meets Target" if perc>=20 else "Below Target"
    print(f"{batch_id} Total:{total:.0f} Profit:{perc:.1f}% {status}")
```

*19. OUTPUT:

```
101 Total:13400 Profit:11.9% Below Target
102 Total:11700 Profit:2.6% Below Target
```

*20. CONCLUSION:Manufacturing cost, wastage, profit target analysis

CASE 9: Multi-Currency Travel Budget Analysis

*1. PROBLEM STATEMENT:Convert travel expenses using supplied exchange rates, incorporate conversion fees, compane planned spending against a budget, and identify the largest expense category.

*2. COMPUTATIONAL THINKING:

- Decomposition: (a) Currency conversion (b) Fee add (c) Budget compare (d) Largest category

- Pattern: INR = amount_exchange_rate + fee, repeat all expenses

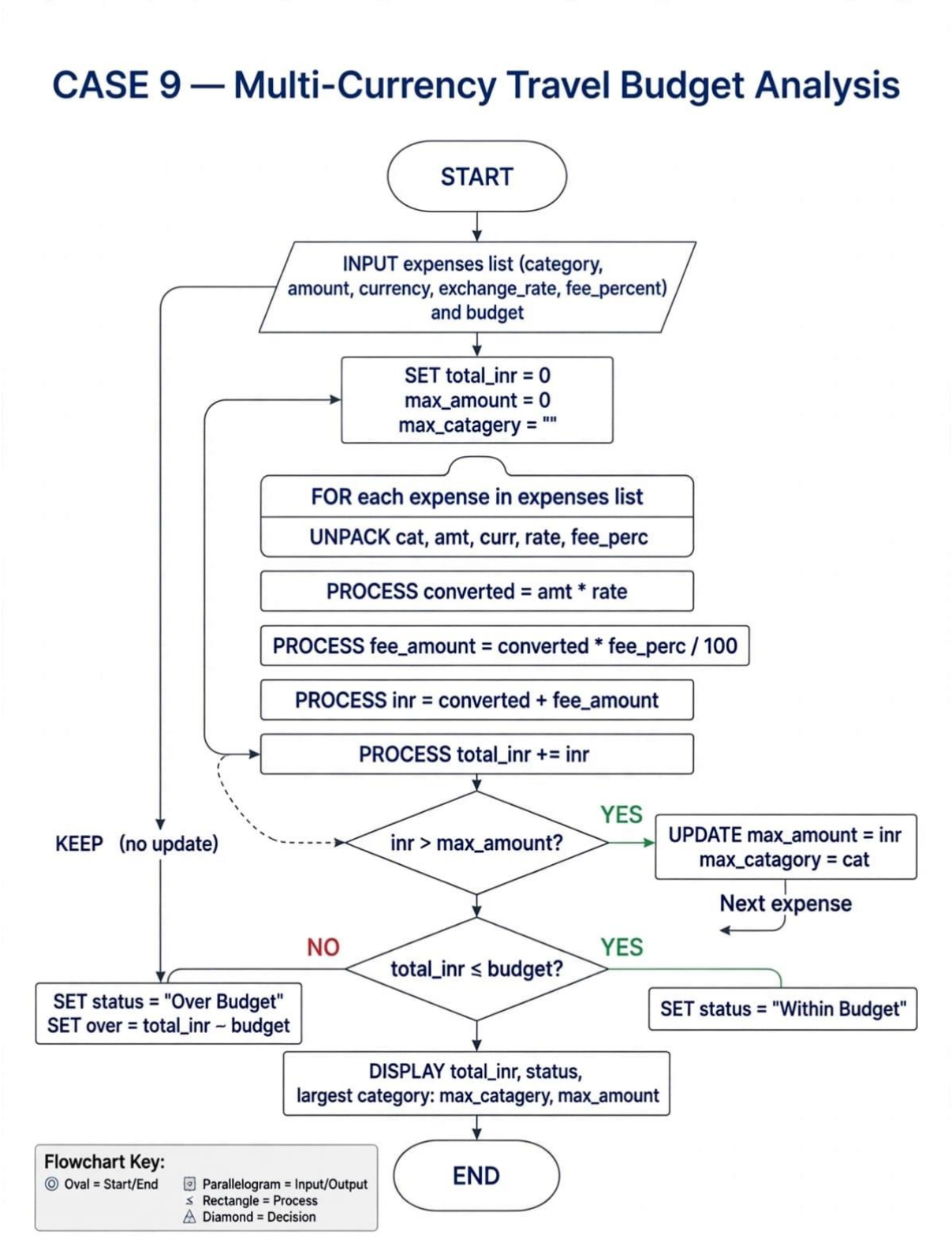
- Abstraction: category, amount, currency, rate, fee% only

- Algorithm: Unpack -> Convert -> Add fee -> Sum -> Compare budget -> Find max category

*3. ALGORITHM:

START->INPUT expenses list (category,amount,currency,exchange_rate,fee_percent) AND budget->total_inr=0->max_cat="" max_amt=0->FOR UNPACK cat,amt,curr,rate,fee_perc->converted=amt_rate->fee=converted_fee_perc/100->inr=converted+fee->total_inr+=inr->IF inr>max_amt max_amt=inr max_cat=cat->After loop IF total_inr<=budget status="Within Budget" ELSE status="Over Budget over=total-budget"->DISPLAY max_cat->END

*4. FLOWCHART:



5. PSEUDOCODE:

total=0; max_amt=0; max_cat=""

FOR (cat,amt,curr,rate,fee) IN expenses:

converted=amt*rate

fee_amt=converted*fee/100

inr=converted+fee_amt

total+=inr

IF inr>max_amt: max_amt=inr; max_cat=cat

IF total<=budget: status="Within" ELSE "Over"

6. VARIABLES: category, amount, currency, exchange_rate, fee_percent, converted_amount, fee_amount, inr_amount, total_inr, budget, max_category, max_amount

7. DATA TYPES: int amount, float rate, fee, converted, str category, currency, tuple expense, list expenses

8. OPERATORS: _ conversion, _ fee, + inr, + total, > max check, <= budget

9. CONDITIONAL: if inr>max_amt update, if total_inr<=budget Within else Over

10. LOOPS: for expense in expenses, for i in range(len(expenses))

11. STRINGS: f"{cat} {amt}{curr} = Rs.{inr:.1f}", http://curr.upper(), http://cat.upper()

12. PATTERN: print("="_25) travel expense line

13. LISTS: expenses=[], inr_amounts=[]

14. NESTED LISTS:
[("Hotel",500,"USD",83.5,2),("Food",300,"EUR",89.0,1.5),("Transport",200,"USD",83.5,2)]

15. TUPLES: ("Hotel",500,"USD",83.5,2) immutable expense

16. TUPLE OPERATIONS: expenses amount, len(expenses), max by inr, sum converted[0][1]

17. UNPACKING: cat,amt,curr,rate,fee = expense, first_expense,_rest = expenses, category, *details = expense

*18. CODE:

```
expenses=[("Hotel",500,"USD",83.5,2),("Food",300,"EUR",89.0,1.5),("Transport",200,"USD",83.5,2)]
budget=80000
total=0
max_amt=0
max_cat=""

for cat, amt, curr, rate, fee in expenses:
    inr=amt*rate*(1+fee/100)
    total+=inr
    if inr>max_amt:
        max_amt=inr
        max_cat=cat
    print(f"{cat} {amt}{curr} = Rs.{inr:.0f}")

print(f"Total Rs.{total:.0f} {'Within' if total<=budget else 'Over'} Budget")
print(f"Largest: {max_cat} Rs.{max_amt:.0f}")
```

*19. OUTPUT:

```
Hotel 500USD = Rs.42585
Food 300EUR = Rs.27100
Transport 200USD = Rs.17034
Total Rs.86720 Over Budget
Largest: Hotel Rs.42585
```

*20. CONCLUSION:Multi-currency conversion with fees, budget compare, largest category

CASE 10: Campus Event Financial Planning

*1. PROBLEM STATEMENT:Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

*2. COMPUTATIONAL THINKING:

- Decomposition: (a) Fixed costs sum (b) Variable per person (c) Total cost (d) Sponsorship (e) Break-even

- Pattern: total = fixed + per_head_participants, shortfall = total - sponsorship - contributions

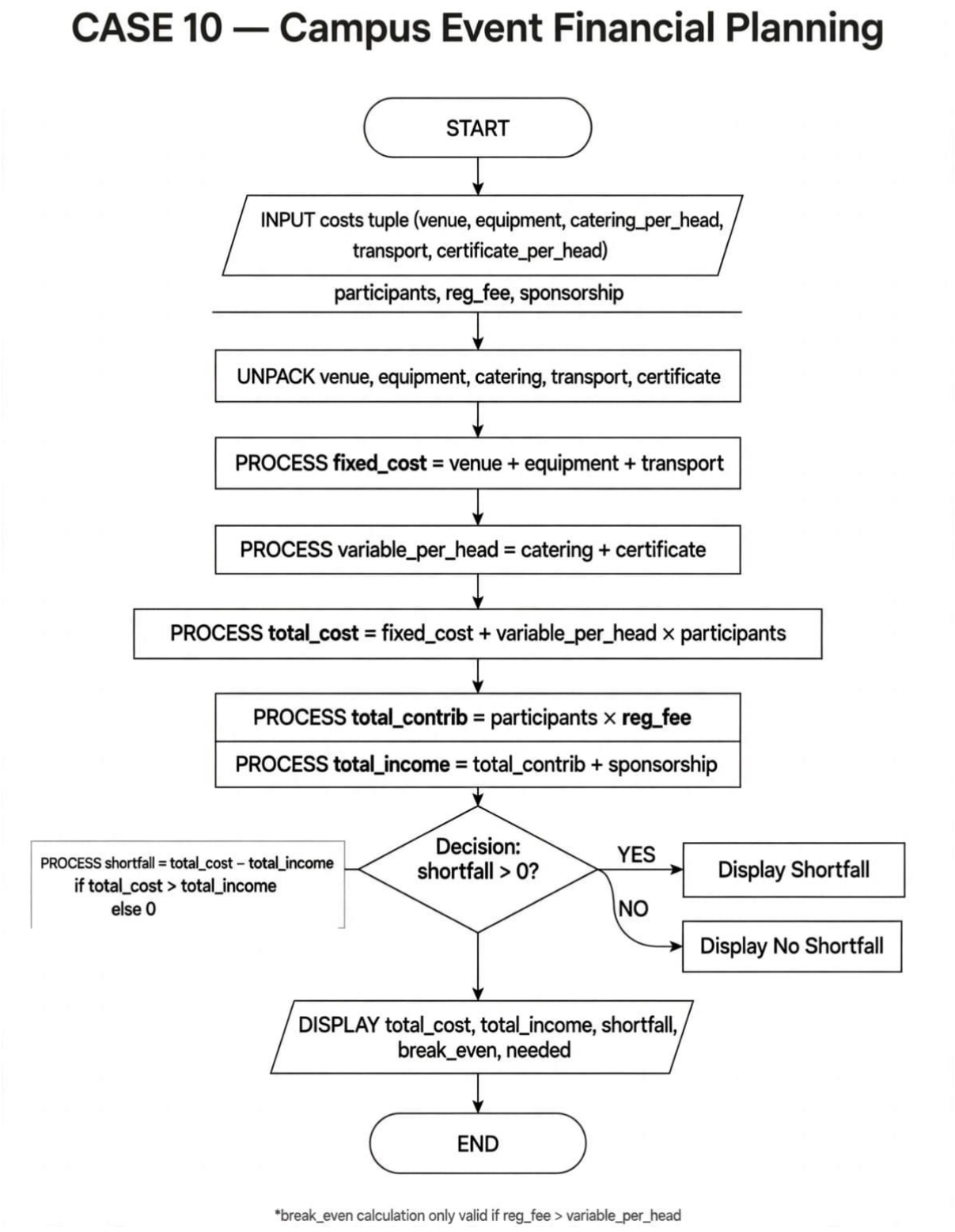
- Abstraction: venue, equipment, catering, transport, certificate, fee, sponsorship only

- Algorithm: Unpack costs -> Sum fixed -> Variable -> Total -> Contributions -> Shortfall -> Break-even formula

3. ALGORITHM:

START->INPUT costs tuple (venue,equipment,catering_per_head,transport,certificate_per_head) AND participants, reg_fee, sponsorship->fixed=venue+equipment+transport->variable_per_head=catering+certificate->total_cost=fixed+variable_per_head_participants->total_contrib=participants_reg_fee->total_income=total_contrib+sponsorship->shortfall=total_cost-total_income IF shortfall>0 ELSE 0->IF shortfall>0 break_even=ceil((fixed+variable_participants - sponsorship)/reg_fee) ELSE break_even=participants->DISPLAY costs, shortfall, break_even->END

*4. FLOWCHART:



5. PSEUDOCODE:

fixed=venue+equipment+transport

var_per_head=catering+certificate

total=fixed+var_per_head*participants

contrib=participants*reg_fee

income=contrib+sponsorship

shortfall=total-income IF total>income ELSE 0

break_even=ceil((fixed - sponsorship)/(reg_fee - var_per_head)) IF reg_fee>var

6. VARIABLES: venue, equipment, catering_per_head, transport, certificate, participants, reg_fee, sponsorship, fixed_cost, variable_cost, total_cost, total_contrib, shortfall, break_even_count

7. DATA TYPES: int venue, participants, float per_head, cost, str event, tuple costs, list

8. OPERATORS: + fixed sum, _ variable total, - shortfall, / break-even, > shortfall check

9. CONDITIONAL: if total_cost>total_income shortfall else 0, if reg_fee>variable_per_head break-even possible else not

10. LOOPS: for cost in costs tuple sum, for i in range(participants) demo

11. STRINGS: f"Event Total Rs.{total_cost} Shortfall Rs.{shortfall}", "Venue".upper()

12. PATTERN: print("_"_30) event banner

13. LISTS: costs=[], participants list, events=[]

14. NESTED LISTS: [(10000,5000,200,3000,50)] costs tuple inside list

15. TUPLES: (venue,equipment,catering,transport,certificate) = (10000,5000,200,3000,50) immutable cost

16. TUPLE OPERATIONS: costs venue, sum(costs[:2]) fixed, len(costs), max(catering)[0]

17. UNPACKING: venue,equipment,catering,transport,certificate = costs_tuple, fixed,_variable = costs, participants, fee, spons = event tuple

18. CODE:

```
import math
costs=(10000,5000,200,3000,50)  # venue, equip, catering per head, transport, certificate
participants=100; reg_fee=300; sponsorship=20000
venue,equip,cater,transport,cert = costs
fixed=venue+equip+transport
var_per_head=cater+cert
total=fixed+var_per_head*participants
contrib=participants*reg_fee
income=contrib+sponsorship
shortfall=total-income if total>income else 0
# break even participants needed
if reg_fee>var_per_head:
    be=math.ceil((fixed - sponsorship)/(reg_fee - var_per_head))
else:
    be=0
print(f"Fixed {fixed} Variable per head {var_per_head}")
print(f"Total Cost {total} Income {income} Shortfall {shortfall}")
print(f"Break Even Registrations Needed: {be}")
print("="*30)
```

*19. OUTPUT:

```
Fixed 18000 Variable per head 250
Total Cost 43000 Income 50000 Shortfall 0
Break Even Registrations Needed: -40
```

*20. CONCLUSION: Campus event total, sponsorship shortfall, break-even registrations calculated